

Canonical source: 00-CHARTER/GUIDES/BUN\_RUNTIME.md

# Æ Orange AI Computer Bun Runtime

## Decision

Æ Orange AI Computer uses Bun for the hot control plane, service management, gateways, benchmarks, and SQLite-backed operational queues. Bun is not a universal rewrite mandate: Rust/Tauri owns the native shell, Python owns proven model and visual workers, and Docker owns isolated infrastructure where replacement has not been proven superior.

The active runtime is Bun 1.3.14. It matched the latest stable Bun release when this contract was measured on 2026-08-27.

## Runtime Laws

1. Hot Orange services run as hidden, detached `Bun.spawn` children with ignored standard I/O. They must not open PowerShell windows or keep their launcher process alive.
2. Independent health probes run concurrently. Dependent start order remains: memory plus Hermes, then OrangeLLM plus Brain MCP.
3. Do not use `--smol` for resident Orange services. Bun documents it as a memory-for-performance trade and the N150 control plane is latency-sensitive.
4. HTTP gateways declare deliberate idle timeouts. Brain MCP and Hermes allow the Bun maximum of 255 seconds for long governed work. Cobra allows 180 seconds for memory/event work. This server ceiling does not override a caller or worker deadline. The fresh Hermes Brain MCP proof completed its harmless read delegation in `11,409.53 ms`; long jobs retain separate task and cancellation contracts.
5. The learning queue uses SQLite WAL, `synchronous=NORMAL`, foreign keys, cached `db.query()` statements, and transactions. This queue is replayable work; immutable receipt truth remains separately hash-chained.
6. Semantic recall uses Qdrant plus Qwen embeddings for dense retrieval and an SSD lexical mirror plus Flux-ledger overlay for exact recall. Payload tokens are computed once per resident process and reused for lexical ranking and reranking.
7. Performance claims require both latency and retrieval-quality proof. A faster result that fails the 23-case corpus is rejected.
8. Profile with Bun's CPU or heap profilers before changing hot code. Profiler overhead is not used as production latency evidence.

---

## Current Proof

Fresh runtime benchmark:

```
10-RECEIPTS/orange5-build/2026-08-27T06-20-11-430Z-bun-runtime-benchmark.json
```

- Six of six runtime endpoints answered.
- Parallel health probing was 2.041 times faster than serial probing.
- Learning queue sustained 1,613.766 operations per second across 300 durable enqueue, lease, and complete operations.
- Hybrid semantic recall completed in 574 ms.

Fresh accepted memory quality benchmark:

```
10-RECEIPTS/orange5-build/2026-08-27T16-42-16-141Z-memory-quality-benchmark.json
```

- 23 of 23 cases passed.
- Hybrid MRR was 0.9058.
- Hybrid p50 was 281 ms and p95 was 445 ms.
- Lexical-only passed 20 of 23; dense-only passed 21 of 23; hybrid passed all 23.
- Three contradiction-debt cases were recorded and resolved by the evidence law.

Fresh Context Crystal quality is scope-bounded: the held-out suite passed 5/5 with minimum `1422.901x` across a 7,056,795-byte corpus. The held-out ratio is a workload-specific operational context result, not a live-turn claim.

The learning-queue focused suite improved from 23.82 seconds before the Bun/SQLite pass to 3.86 seconds after removing repeated schema, statement-compilation, and post-write lookup work.

CPU profiles before and after the semantic-index change show tokenization falling from 561.7 ms to 175.9 ms and the lexical-candidate stage falling from 619.0 ms to 319.2 ms. The final profiled benchmark remained green; its profile is:

```
10-RECEIPTS/orange5-build/profiles/CPU.28574002566.11168.md
```

---

## Commands

```
bun scripts/orange5-runtime-services.mjs status
bun run bench:bun-runtime
bun run bench:memory-quality
bun --cpu-prof-md --cpu-prof-dir 10-RECEIPTS/orange5-build/profiles scripts/bun-runtime-
benchmark.mjs
```

---

## Primary References

- Bun releases: <https://github.com/oven-sh/bun/releases>
- Bun benchmarking and profiling: <https://bun.sh/docs/project/benchmarking>
- Bun SQLite: <https://bun.sh/docs/runtime/sqlite>
- Bun HTTP server: <https://bun.sh/docs/runtime/http/server>
- Bun child processes: <https://bun.sh/docs/runtime/child-process>
- Bun tests: <https://bun.sh/docs/test>
- SQLite PRAGMA guidance: <https://www.sqlite.org/pragma.html>

---

## Next Optimization Threshold

Do not tune by instinct. Reprofile only when a fresh non-profiled run breaches one of these limits:

- Any local health endpoint exceeds 250 ms p95.
- Hybrid memory recall exceeds 1,000 ms p95 or falls below 0.80 MRR.
- Learning queue falls below 1,000 operations per second on the N150.
- A resident Bun organ grows without bound across a 24-hour soak.

At that point, optimize the measured hot function and rerun both performance and behavioral proof before promotion.